

BUG-1 fix evidence — telemetry-overview client/server model drift

Revision: 1 Last modified: 2026-07-10T02:40:00Z

Scope: clients/ota-manager/ (src) only. Stream TEL (T1/main - claude1).

1. The real server contract (established as FACT, not guessed)

server/internal/api/handlers_telemetry.go:56-64 (TelemetryOverview struct) and :168-193 (handleTelemetryOverview) — the body of GET / telemetry/overview — is:

```
type TelemetryOverview struct {
    EventCounts map[string]int64 `json:"event_counts"`
    Total       int64      `json:"total"`
    FailureRate float64     `json:"failure_rate"`
    ByState     map[string]int64 `json:"by_state"`
}
```

- event_counts — cumulative telemetry-event counts by type, across all history (six known event types from submodules/ota-protocol/ enums.go: download_started, installing, installed, verifying, success, failure).
- total — sum of event_counts.
- failure_rate — failure / (success + failure) among terminal outcomes.
- by_state — **fleet device count keyed by each device's LAST-KNOWN update state** (server/internal/store/memory.go:562-576 DeviceStateCounts, postgres.go:749+ equivalent). One bucket per registered device (current state, not cumulative). Possible keys, traced to source:
 - "idle" — registration default (handlers_device.go:62,215).

- "unknown" — defensive bucket for an empty/never-set state (`memory.go:570-572`).
- "download_started" | "installing" | "installed" | "verifying" — in-flight update states. `handlers_client.go:207` sets `dev.UpdateState = string(ev.Event)` verbatim from the device's last reported `TelemetryEvent`, so these four non-terminal event names are real `by_state` keys.
- "success" — last update completed successfully (terminal, not pending).
- "failure" — last known state is a failed update (terminal).

There is **no deployment data anywhere in this response** — active deployments has no source at this endpoint.

2. The client-side defect (pre-fix)

- `clients/ota-manager/src/lib/api-client.ts` declared the client `TelemetryOverview` TypeScript interface as the **dashboard's own camelCase shape** — `{totalDevices, activeDeployments, pendingUpdates, failedDevices}` — not the real wire shape above. A type-level fabrication.
- `clients/ota-manager/src/hooks/use-devices.ts`'s raw `useTelemetryOverview` called `apiGet<TelemetryOverview>('/telemetry/overview')`, casting the *real* JSON response (`event_counts/total/failure_rate/by_state`) to that fabricated type with **zero runtime mapping**.
- `clients/ota-manager/src/hooks/useTelemetryOverview.ts` (the file `dashboard-page.tsx` actually imports) was a **bare re-export**: `export { useTelemetryOverview } from './use-devices';` — no adapter at all, unlike the sibling `useDevices.ts` which DOES adapt the raw device list into a view model.
- Net effect: `dashboard-page.tsx` read `overview.totalDevices`, `overview.activeDeployments`, `overview.pendingUpdates`, `overview.failedDevices` off an object that actually only had `event_counts/total/failure_rate/by_state` — every field was undefined, and every stat card fell through its `?? 0` fallback and rendered `0` regardless of the real fleet state.

3. Field-by-field mapping implemented

All in `clients/ota-manager/src/hooks/useTelemetryOverview.ts` (full rewrite, extensively commented in-source with the same citations as this document):

Dashboard stat	Source	Derivation	Status
<code>totalDevices</code>	<code>/telemetry/overview by_state</code>	sum of every <code>by_state</code> value	real-derived
<code>pendingUpdates</code>	<code>/telemetry/overview by_state</code>	<code>by_state["download_started"] + ["installing"] + ["installed"] + ["verifying"]</code> (the four non-terminal states)	real-derived
<code>failedDevices</code>	<code>/telemetry/overview by_state</code>	<code>by_state["failure"]</code> — current-state count	real-derived
<code>activeDeployments</code>	<code>GET /deployments</code> (<code>useDeployments()</code>)	<code>items.length</code>	real, fetched from the correct endpoint — no stat is n/a

No stat was marked n/a: `activeDeployments` had no source at `/telemetry/overview`, but `server/internal/api/handlers_deployment.go:106-117` (`handleListDeployments`) calls `s.repo.ListActiveDeployments` **unconditionally** (no status filter is read or applied anywhere in that handler) — so every item the real `GET /deployments` endpoint returns is already an active deployment, and `items.length` is a genuine, non-fabricated count. The adapter now composes both react-query hooks (`use-devices.ts`'s `useTelemetryOverview` + `use-deployments.ts`'s `useDeployments`) and gates combined `isLoading/isError` so the dashboard doesn't flash a transient `activeDeployments = 0` while the second query is still in flight.

I also corrected `clients/ota-manager/src/lib/api-client.ts`'s `TelemetryOverview` interface to the real wire shape (it was the root type-level fabrication) and renamed the dashboard-facing camelCase shape to a new `TelemetryOverviewView` interface local to the adapter file. Nothing else imports the old (wrong) `TelemetryOverview` type, so this was safe within scope.

4. Files changed

- `clients/ota-manager/src/lib/api-client.ts` — `TelemetryOverview` interface corrected to the real server shape (`event_counts/total/failure_rate/by_state`), with a citation comment.
- `clients/ota-manager/src/hooks/useTelemetryOverview.ts` — full rewrite: real adapter mapping (previously a bare re-export), heavily commented with `file:line` citations into `server/internal/api/handlers_telemetry.go`, `handlers_deployment.go`, `handlers_device.go`, `handlers_client.go`, `store/memory.go`, and `submodules/ota-protocol/enums.go`.
- `clients/ota-manager/src/__tests__/use-telemetry-overview.test.tsx` — NEW. Real-shaped-fixture unit test (5 cases) exercising the actual (unmocked) `use-devices.ts` + `use-deployments.ts` hooks through the adapter, mocking only `apiGet` at the transport boundary.
- `clients/ota-manager/src/__tests__/dashboard.test.tsx` — mock boundary fixed: was mocking `@/hooks/use-devices` (a module `dashboard-page.tsx` doesn't even import) with an already-mapped `camelCase` fixture, which happened to pass only because the old adapter was a bare re-export of that same module. Now mocks `@/hooks/useTelemetryOverview` (the module `dashboard-page.tsx` actually imports), keeping this test a pure “dashboard renders given an already-mapped view model” component test — the mapping logic itself is covered by the new dedicated test file.

5. RED → GREEN evidence (§11.4.115)

RED — captured against the OLD code (`export { useTelemetryOverview } from './use-devices';`, no mapping), running the NEW test file `use-telemetry-overview.test.tsx`:

```
FAIL src/__tests__/use-telemetry-overview.test.tsx > ... >
derives totalDevices ...
FAIL src/__tests__/use-telemetry-overview.test.tsx > ... >
derives pendingUpdates ...
AssertionError: expected undefined to be 8
FAIL src/__tests__/use-telemetry-overview.test.tsx > ... >
derives failedDevices ...
AssertionError: expected undefined to be 3
FAIL src/__tests__/use-telemetry-overview.test.tsx > ... >
derives activeDeployments ...
AssertionError: expected undefined to be 3
```

```
FAIL  src/__tests__/use-telemetry-overview.test.tsx > ... > never
surfaces the raw server field names ...
AssertionError: expected { event_counts: { ...(6) }, ...(3) } to not
have property "event_counts"
```

```
Test Files  1 failed (1)
Tests       5 failed (5)
```

All 5 failures reproduce the exact real-world defect: the hook's data was the raw server object (event_counts/total/failure_rate/by_state) with every dashboard-expected camelCase field undefined.

GREEN — same test file, same fixtures, after the fix:

```
RUN  v4.1.9
```

```
Test Files  1 passed (1)
Tests       5 passed (5)
```

6. Full verification numbers

- `npx tsc --noEmit` → **0 errors** (clean).
- `npx vitest run` (full suite) → **11 test files passed, 49 tests passed** (0 failed) — up from 44 tests pre-fix (+5 new), no regressions in any other suite (dashboard.test.tsx, api-client-baseurl.test.ts, auth-guard.test.tsx, data-table.test.tsx, login-form.test.tsx, sidebar.test.tsx, features/deployments.test.tsx, features/devices.test.tsx, features/releases.test.tsx, ui-store.test.ts).

7. Related finding — NOT fixed, out of scope (honest disclosure, §11.4.6)

While tracing the real server contract I found a second, separate type-level mismatch in the same file: `clients/ota-manager/src/lib/api-client.ts`'s `TelemetryHistory` and `DeploymentList` interfaces also declare `{items, total, cursor?}`, but the real server shapes (`handlers_telemetry.go:35-39` `TelemetryHistory`, `wire.go:187-190` `DeploymentList`) are `{..., items, next_cursor}` (no total, no cursor, paginated via `next_cursor`). This did NOT block this fix — the `activeDeployments` derivation only reads `.items.length`, which exists under both the correct and the mistyped shape — but it is a live, undisclosed defect for any UI code that reads `.total` or `.cursor` off `useDeviceTelemetry` / `useDeployments` (pagination “load more”

affordances would silently read undefined). Flagging for a separate workable item; not fixed here to stay within the disclosed BUG-1 scope and avoid uncontrolled blast radius into other streams' territory.